4. Построение иерархической структуры работ (WBS)

Планирование содержания занимает центральное место в управлении программным проектом. После определения требований и аинтересованных сторон необходимо разложить весь объём работ на управляемые части, чтобы оценить трудоёмкость, распределить ответственность и построить календарный план. Основным инструментом такой декомпозиции является иерархическая структура работ (Work Breakdown Structure, WBS).

WBS связывает содержание проекта с работами, которые должны быть выполнены, и служит основой для оценки сроков, бюджета и рисков. Пропущенные на этом этапе задачи приводят к завышенным оценкам, срыву сроков и конфликтам при приёмке результатов. Поэтому построение WBS выполняется до детального планирования расписания.

Объектом исследования является учебный проект SurveyFlow — веб-приложение для создания онлайн-опросов и анализа ответов в реальном времени.

Предметом исследования является содержание проекта SurveyFlow и его декомпозиция до уровня рабочих пакетов.

Цель работы — построить иерархическую структуру работ проекта SurveyFlow до уровня рабочих пакетов и подготовить фрагмент WBS Dictionary.

Для достижения поставленной цели необходимо решить следующие задачи:
– определить основные результаты (deliverables) верхнего уровня;
– декомпозировать каждый результат до рабочих пакетов с оценкой 8–80 часов;
– оформить WBS в виде схемы и нумерованного перечня;
– составить WBS Dictionary для пяти рабочих пакетов;

– проверить соблюдение правил 100% и 8/80.

1 Теоретические основы построения WBS

1.1 Содержание проекта и иерархическая структура работ

Управление содержанием проекта начинается с разграничения двух понятий. Содержание продукта (Product Scope) описывает свойства, функции и характеристики результата, который получит заказчик. Содержание проекта (Project Scope) определяет работы, необходимые для создания этого результата. WBS относится к управлению содержанием проекта и фиксирует не только конечный продукт, но и все промежуточные результаты.
Согласно PMBOK, иерархическая структура работ (Work Breakdown Structure, WBS) — это иерархическая декомпозиция всего объёма работ, необходимых для достижения целей проекта и создания требуемых результатов. WBS представляет проект в виде дерева: на верхнем уровне находится сам проект, на нижних — его крупные результаты и всё более мелкие элементы.
Элемент WBS — это работа, результат или их совокупность, выделенная в структуре. Элемент нижнего уровня, для которого можно оценить трудоёмкость, назначить исполнителя и определить критерии приёмки, называется рабочим пакетом. WBS ориентирована на результаты, а не на последовательность действий: она отвечает на вопрос «что должно быть создано», а не «в каком порядке это делать».
Элементы WBS нумеруются иерархически: 1.0 — проект, 1.1–1.7 — результаты второго уровня, 1.1.1–1.7.3 — рабочие пакеты, 1.1.1.1 и далее — подзадачи. Такая нумерация позволяет однозначно ссылаться на элемент при планировании, оценке и контроле. На основе WBS строятся календарный план, смета, матрица ответственности и система отчётности.

1.2 Правило 100% и принципы декомпозиции

Ключевым требованием к WBS является правило 100%: сумма дочерних элементов каждого уровня должна полностью покрывать содержание родительского элемента — без пропусков и без дублирования. Если часть работ не отражена в структуре, она не будет запланирована и оценена; если работы учтены дважды, возникнут завышенные оценки и конфликты при распределении ответственности.
Декомпозиция выполняется сверху вниз и подчиняется принципу MECE (mutually exclusive, collectively exhaustive): элементы одного уровня не пересекаются по содержанию и вместе исчерпывают родительский элемент. У каждого элемента должен быть только один родитель, иначе нарушается однозначность структуры и отчётности.
Глубина декомпозиции определяется возможностью управлять работой. Для учебного проекта достаточно трёх-четырёх уровней: проект, крупные результаты, рабочие пакеты и подзадачи. Декомпозиция прекращается, когда пакет можно оценить, поручить одному исполнителю и проверить по измеримым критериям.
Для оценки размера пакета используется правило 8/80: трудоёмкость рабочего пакета должна составлять от 8 до 80 часов. В обновлённой WBS принят диапазон 8–24 часа: этого достаточно для оценки, назначения одного ответственного и проверки за спринт.

1.3 Методы построения WBS и правило 8/80

WBS можно строить разными способами: по фазам жизненного цикла, по основным результатам (deliverables) и по функциональным областям. При построении по фазам верхний уровень образуют этапы (анализ, проектирование, разработка, тестирование), при построении по результатам — создаваемые артефакты, при функциональном подходе — направления работ (backend, frontend, тестирование).
В обновлённой версии от 27.10.2026 выбран модульный подход: верхний уровень образуют управление проектом, инфраструктура и DevOps, аналитика и дизайн, техническая разработка и релиз.

Внутри технической разработки выделены функциональные модули «Авторизация», «Личный кабинет», «Конструктор опросов», «Прохождение опроса», «Живая статистика» и «Экспорт PDF/SPSS». В каждом модуле параллельно работают аналитик, backend, frontend и QA.

1.4 WBS Dictionary

WBS Dictionary (словарь WBS) — документ, который детализирует содержание каждого рабочего пакета. Для каждого элемента указываются код, наименование, описание работ, входные данные и результат, критерии приёмки, ответственный, оценка трудоёмкости и зависимости от других пакетов.
Словарь устраняет неоднозначность формулировок: разные участники одинаково понимают, что входит в пакет и когда он считается завершённым. На основе словаря проводятся приёмка работ, контроль качества и оценка выполнения. В данной работе словарь составлен для пяти ключевых пакетов, по одному от каждого участника команды.

1.5 WBS и Product Backlog

В гибких методологиях содержание часто представляется не деревом, а Product Backlog — упорядоченным списком пользовательских историй. WBS и backlog решают разные задачи: WBS фиксирует полный объём работ и его иерархию, backlog определяет приоритеты и порядок реализации в спринтах.
WBS более устойчива и ориентирована на результаты, backlog постоянно уточняется и пополняется. На практике инструменты дополняют друг друга: WBS используется для общей оценки и контроля содержания, backlog — для планирования итераций. В учебном проекте рабочие пакеты WBS связываются с пользовательскими историями, а оценки пакетов служат ориентиром при планировании спринтов.

2 Построение WBS проекта SurveyFlow

2.1 Краткая характеристика проекта

SurveyFlow — веб-приложение для быстрого создания опросов и анализа ответов в реальном времени. Пользователь регистрируется, создаёт опрос из пяти типов вопросов, настраивает анонимность и повторное прохождение, распространяет публичную ссылку. Результаты обновляются без перезагрузки страницы, визуализируются диаграммами, отчёт выгружается в PDF и CSV для SPSS.

Срок проекта — 21.10.2026–20.12.2026 (9 недель). Команда — 5 человек, по 72 часа на каждого, общий бюджет — 360 часов. Стек: Vue.js, TypeScript, Pinia, Node.js, Express.js, Socket.IO, MariaDB, PDFKit, Git.

Участник	Роль	Зона ответственности	Часы
Зрожаев А.В.	PM + Frontend	Управление, простые UI-задачи	72
Гетьман М.Е.	Lead Dev + Frontend	Архитектура, сложный frontend, review	72
Серкин К.А.	Backend + DevOps	API, БД, CI/CD	72
Игнатчик В.В.	Аналитик + UI/UX	Требования, сценарии, прототипы	72
Царева Е.М.	QA + QA/DevOps	Тест-планы, проверки, регрессия	72
Итого	5 ролей	—	360

2.2 Принцип построения WBS

WBS построена по функциональным модулям. Уровень 3 — готовый модуль, уровень 4 — конкретные задачи с тегом роли. Внутри каждого модуля аналитик, backend, frontend и QA работают параллельно: аналитик готовит требования и критерии, backend делает API и БД, frontend — интерфейс и интеграцию, QA сразу пишет чек-лист и тест-кейсы.

Обязательные правила: у каждой задачи нижнего уровня есть тег роли в квадратных скобках; в каждом модуле присутствуют аналитика, разработка, интеграция и тестирование; QA и аналитик участвуют во всех модулях и имеют значимый объём работ; PM совмещает управление с frontend; трудоёмкость задачи — 8–80 часов.

2.3 Верхний уровень WBS

Код	Блок	Содержание	Критерий завершения	Часы
1	Управление проектом	Инициация, план, мониторинг, риски, EVM, закрытие	Устав, WBS, календарный план, портфолио	40
2	Инфраструктура и DevOps	Git, Git-flow, CI/CD, тестовый стенд, мониторинг	Репозиторий, pipeline, тестовый сервер	56
3	Аналитика и дизайн	Use Cases, User Stories, DoD, UI/UX-прототипы	Спецификация, прототипы, критерии	24
4	Техническая разработка	Шесть функциональных модулей	Работающие модули, интеграция, тесты	216
5	Релиз, приёмка и защита	Регрессия, нагрузка, приёмка, демо, защита	Приёмочный отчёт, демо, защита	24

2.4 Модульная структура WBS

Ниже — упрощённая декомпозиция: 5 блоков, 6 модулей и 4 задачи в каждом модуле. Этого достаточно, чтобы видеть полный контур работ, роли и часы без избыточной детализации.

Код	Элемент WBS и состав работ	Роль	Часы	Результат
1	**Управление проектом**	**—**	**40**	**Командный менеджмент и мониторинг**
1.1	Инициация: устав, WBS, PERT, диаграмма Ганта, baseline	[PM]	16	Устав, WBS, календарный план, 5 вех
1.2	Мониторинг: Scrum-события, риски, EVM, Change Request	[PM / Вся команда]	16	Протоколы, реестр рисков, EVM-отчёт
1.3	Закрытие: портфолио артефактов, презентация, защита	[PM]	8	Портфолио, презентация
2	**Инфраструктура и DevOps**	**—**	**56**	**Промышленная среда разработки**
2.1	Git-репозиторий, Git-flow, PR-review	[Lead Dev / PM]	8	Репозиторий, регламент ветвления
2.2	CI: автоматическая сборка и линтеры	[Lead Dev]	8	CI-pipeline сборки
2.3	CI-тесты: модульные и интеграционные	[Lead Dev]	8	Автозапуск тестов
2.4	CD: деплой на тестовый сервер, мониторинг	[Backend]	16	Тестовый стенд, мониторинг
2.5	Тестовый контур и локальная СУБД	[QA]	8	Изолированная среда тестов
2.6	Резервное копирование и мониторинг	[Lead Dev]	8	Регламент резервного копирования

3	**Аналитика и дизайн**	**—**	**24**	**Требования и проектирование интерфейса**
3.1	Use Cases, User Stories, DoD, критерии приёмки	[Аналитик]	16	Спецификация, DoD, критерии
3.2	UI/UX-прототипы ключевых экранов, дизайн-ревью	[Аналитик]	8	Прототипы, замечания
4	**Техническая разработка**	**—**	**216**	**Шесть функциональных модулей**
4.1	**Модуль «Авторизация и профиль»**	**—**	**32**	**Регистрация, вход, профиль**
4.1.1	Сценарии входа, регистрации, восстановления пароля	[Аналитик]	8	Use Cases, критерии
4.1.2	API авторизации и профиля, таблицы пользователей	[Backend]	8	authController.ts, routes/auth.ts
4.1.3	Вёрстка LoginView.vue и RegisterView.vue, интеграция	[PM / Frontend]	8	Экраны входа и регистрации
4.1.4	Чек-лист, тест-кейсы, баг-репорты	[QA]	8	Отчёт тестирования модуля
4.2	**Модуль «Личный кабинет»**	**—**	**32**	**Список опросов, поиск, фильтры**
4.2.1	Сценарии кабинета, поиск, фильтры, пагинация	[Аналитик]	8	Use Cases, критерии
4.2.2	API списка, удаления и статусов опросов	[Backend]	8	surveyController.ts
4.2.3	Карточки, фильтры, пагинация, интеграция	[PM / Frontend]	8	UI кабинета
4.2.4	Чек-лист, тест-кейсы, баг-репорты	[QA]	8	Отчёт тестирования модуля
4.3	**Модуль «Конструктор опросов»**	**—**	**40**	**Создание и редактирование опроса**
4.3.1	Сценарии конструктора, 5 типов вопросов	[Аналитик]	8	Use Cases, правила валидации
4.3.2	API CRUD опросов и вопросов	[Backend]	16	surveyController.ts, routes/survey.ts
4.3.3	UI карточки вопроса, настройки, интеграция	[PM / Frontend]	8	SurveyBuilder.vue
4.3.4	Чек-лист, тест-кейсы, баг-репорты	[QA]	8	Отчёт тестирования модуля
4.4	**Модуль «Прохождение опроса»**	**—**	**32**	**Публичная ссылка, валидация, отправка**
4.4.1	Сценарии прохождения, анонимность, повтор	[Аналитик]	8	Use Cases, критерии
4.4.2	API публичной ссылки и приёма ответов	[Backend]	8	routes/survey.ts, validators.ts

4.4.3	UI SurveyView.vue и интеграция с API	[Frontend]	8	Рабочее прохождение опроса
4.4.4	Чек-лист, тест-кейсы, баг-репорты	[QA]	8	Отчёт тестирования модуля
4.5	**Модуль «Живая статистика»**	**—**	**40**	**Socket.IO, графики, метрики**
4.5.1	Сценарии live-статистики, метрики	[Аналитик]	8	Use Cases, метрики
4.5.2	Socket.IO, агрегация, statsCache	[Backend]	8	surveySocket.ts, config/socket.ts
4.5.3	Графики и real-time интеграция	[Frontend]	16	Живые диаграммы
4.5.4	Чек-лист, тест-кейсы, баг-репорты	[QA]	8	Отчёт тестирования модуля
4.6	**Модуль «Экспорт PDF/SPSS»**	**—**	**40**	**PDF-отчёт и CSV для SPSS**
4.6.1	Сценарии экспорта, форматы, требования к отчёту	[Аналитик]	8	Спецификация экспорта
4.6.2	PDF-сервис и CSV для SPSS	[Backend]	8	pdfExportService.ts, spssExportService.ts
4.6.3	Кнопки и статусы экспорта, интеграция	[Frontend]	16	UI экспорта
4.6.4	Чек-лист, тест-кейсы, баг-репорты	[QA]	8	Отчёт тестирования модуля
5	**Релиз, приёмка и защита**	**—**	**24**	**Финальная проверка и защита**
5.1	Регрессия, нагрузка, безопасность, приёмочный отчёт	[QA]	16	Регрессионный отчёт, акт приёмки
5.2	Демо, презентация, защита проекта	[PM]	8	Демо и защита

Рисунок 1 – Модульная схема WBS проекта SurveyFlow (упрощённая версия от 27.10.2026)

2.5 Словарь WBS

Словарь составлен для пяти ключевых пакетов — по одному от каждой роли.

Код и пакет	Ответственный	Описание работ	Входы и результат	Критерии приёмки
1.1 / Инициация и планирование	Зрожаев А.В. PM 16 ч	Устав, WBS, PERT, диаграмма Ганта, базовое расписание, 5 вех.	Вход: требования, команда. Результат: устав, WBS, календарный план.	1) 5 вех; 2) сумма 360 ч; 3) baseline зафиксирован.
3.1 / Требования и DoD	Игнатчик В.В. Аналитик 16 ч	Use Cases, User Stories для шести модулей, DoD и критерии приёмки.	Вход: устав, аналоги. Результат: спецификация, User Stories.	1) покрыты 6 модулей; 2) у каждой истории есть критерии; 3) INVEST.

4.1.2 / API авторизации и профиля	Серкин К.А. Backend 8 ч	Регистрация, вход, восстановление пароля, таблицы пользователей.	Вход: схема БД, требования. Результат: authController.ts, routes/auth.ts.	1) вход и регистрация; 2) аутентификация; 3) пароль в виде хеша.
4.5.3 / Графики и real-time	Гетьман М.Е. Lead Dev 16 ч	Диаграммы результатов, обновление через Socket.IO, состояния загрузки.	Вход: API статистики, макеты. Результат: SurveyResults.vue, графики.	1) обновление ≤ 3 с; 2) столбчатые и круговые диаграммы; 3) ошибки обработаны.
4.1.4 / QA-чек-лист авторизации	Царева Е.М. QA 8 ч	Чек-лист и тест-кейсы для входа, регистрации и восстановления пароля.	Вход: критерии приёмки, API. Результат: чек-лист, тест-кейсы, баги.	1) позитивные и негативные сценарии; 2) баги заведены; 3) исправления проверены.

2.6 Проверка правила 100% и правила 8/80

Суммы часов по блокам и модулям полностью покрывают проект: 360 часов, по 72 часа на каждую роль.

Блок / модуль	Часы	Доля	Проверка
1. Управление проектом	40	11,1%	16 + 16 + 8 = 40
2. Инфраструктура и DevOps	56	15,6%	8 + 8 + 8 + 16 + 8 + 8 = 56
3. Аналитика и дизайн	24	6,7%	16 + 8 = 24
4.1 Авторизация и профиль	32	8,9%	4 × 8 = 32
4.2 Личный кабинет	32	8,9%	4 × 8 = 32
4.3 Конструктор опросов	40	11,1%	8 + 16 + 8 + 8 = 40
4.4 Прохождение опроса	32	8,9%	4 × 8 = 32
4.5 Живая статистика	40	11,1%	8 + 8 + 16 + 8 = 40
4.6 Экспорт PDF/SPSS	40	11,1%	8 + 8 + 16 + 8 = 40
5. Релиз и защита	24	6,7%	16 + 8 = 24
Итого	360	100,0%	100% работ учтено

Правило 8/80 выполнено: минимальная задача — 8 часов, максимальная — 16 часов. Суммы по ролям: PM — 72, Lead Dev — 72, Backend — 72, Аналитик — 72, QA — 72; итого 360.

2.7 Распределение трудоёмкости по ролям

Участник	Роль	Состав часов	Часы	Доля
Зрожаев А.В.	PM + Frontend	40 управление + 24 frontend + 8 приёмка	72	20,0%

Гетьман М.Е.	Lead Dev + Frontend	32 DevOps + 40 frontend/review	72	20,0%
Серкин К.А.	Backend + DevOps	16 DevOps + 56 API/БД	72	20,0%
Игнатчик В.В.	Аналитик + UI/UX	24 аналитика + 48 модульные сценарии	72	20,0%
Царева Е.М.	QA + QA/DevOps	8 DevOps + 48 модульные тесты + 16 релиз	72	20,0%
Итого	5 ролей	—	360	100,0%

Все роли равноценны: по 72 часа. Аналитик и QA имеют по 72 часа и присутствуют в каждом модуле: аналитик — сценарии и критерии, QA — чек-листы, тесты и баг-репорты. PM не перегружен: 40 часов управления, 24 часа frontend, 8 часов приёмки.

2.8 Вывод

Построена упрощённая модульная WBS проекта SurveyFlow: 5 блоков, 6 модулей, 4 задачи в каждом модуле. Общая трудоёмкость — 360 часов, по 72 часа на каждую роль. Все задачи укладываются в правило 8/80, аналитик и QA участвуют во всех модулях, PM совмещает управление с frontend. Структура готова к использованию в Git и спринтах.